PEMROGRAMAN BERBASIS FRAMEWORK
JOBSHEET 11

NAMA : Muhammad Fahreza Rohmansyah
KELAS : 3E
NO. ABSEN : 15
NIM : 2341720151

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Link Github:

https://github.com/rezfahreza/PBF_2026.git

Praktikum

1. Membuat Dynamic Route

- Modifikasi file view/produk/index.tsx

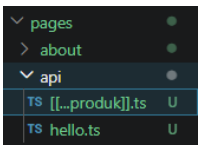
```
my-app > src > view > produk > index.tsx > ...
import styles from "../../pages/produk/product.module.scss";
import Link from "next/link";
import { ProductType } from "@/type/product.type";

const TampilanProduk = ({products}: {products: ProductType[]}) => {
    return (
        <div className={styles.produk}>
            <h1 className={styles.produk__title}>Daftar Produk</h1>
            <div className={styles.produk__content}>
                {products.length > 0 ? (
                <>
                    {products.map((products: ProductType) => (
                        <Link href={'/produk/${products.id}'} key={products.id} className={styles.produk__content__item}>
                            <div className={styles.produk__content__item__image}>
                                <img src={products.image} alt={products.name} width={200} />
                            </div>
                            <h4 className={styles.produk__content__item__name}>
                                {products.name}
                            </h4>
                            <p className={styles.produk__content__item__category}>
                                {products.category}
                            </p>
                            <p className={styles.produk__content__item__price}>
                                Rp {products.price.toLocaleString()}
                            </p>
                        </Link>
                    ))}
                </>
                ) : (
                <div className={styles.produk__content__skeleton}>
                    <div className={styles.produk__content__skeleton__image}></div>
                    <div className={styles.produk__content__skeleton__name}></div>
                    <div className={styles.produk__content__skeleton__category}></div>
                    <div className={styles.produk__content__skeleton__price}></div>
                </div>
                )}
            </div>
        </div>
    );
};

export default TampilanProduk;
```

2. Implementasi CSR (Client Rendering)

- Modifikasi file pages/products/[id].tsx

```
JS_11 > my-app > src > pages > produk > [id].tsx > default
import fetcher from "@/utils/swr/fetcher";
import { useRouter } from "next/router";
import useSWR from "swr";

const HalamanProduk = () => {
    //const router = useRouter();
    //console.log(Router);
    const {query} = useRouter();
    const {data, error,isLoading} = useSWR(`/api/products/${query.id}`, fetcher);
    return (
        <div>
            <h1>Halaman Produk</h1>
            <p>Produk: {query.id}</p>
        </div>
    )
}

export default HalamanProduk;
```

- Ubah nama file api/produk.ts menjadi [[...product]].ts



- Modifikasi servicefirebase.tsx

```
my-app > src > utils > db > TS servicefirebase.ts > retrieveDataById
import { getFirestore, collection, getDocs, Firestore, getDoc, doc } from "firebase/firestore";
import app from "./firebase";

const db = getFirestore(app);

export async function retrieveProducts(collectionName: string) {
    const snapshot = await getDocs(collection(db, collectionName));
    const data = snapshot.docs.map((doc) => ({
        id: doc.id,
        ...doc.data(),
    }));
    return data;
}

export async function retrieveDataById(collectionName: string, id: string) {
    const snapshot = await getDoc(doc(db, collectionName, id));
    const data = snapshot.data();
    return data;
}
```

- Modifikasi [[...product]].ts

```
my-app > src > pages > api > TS [[...produk]].ts > handler
import type { NextApiRequest, NextApiResponse } from 'next';
import { retrieveDataById, retrieveProducts } from '../../utils/db/servicefirebase';

type Data = {
    status: boolean;
    status_code: number;
    data: any;
};

export default async function handler(
    req: NextApiRequest,
    res: NextApiResponse<Data>
) {
    if (req.query.produk![1]) {
        const data = await retrieveDataById('products', req.query.produk![1] as string);
        res.status(200).json({
            status: true,
            status_code: 200,
            data: data,
        });
    } else {
        const data = await retrieveProducts('products');
        res.status(200).json({
            status: true,
            status_code: 200,
            data: data,
        });
    }
}
```

- Buat fil styling di views/DetailProduct/detailProduct.module.scss

```
JS_11 > my-app > src > view > DetailProduct > detailProduct.module.scss > {
.produkdetail {
  &__name {
    margin-bottom: 12px;
    color: #222;
  }

  &__category {
    font-size: 16px;
    color: #666;
    margin-bottom: 16px;
  }

  &__price {
    font-size: 22px;
    font-weight: bold;
    color: #FF5722;
    margin-bottom: 20px;
  }

  &__button {
    width: fit-content;
    padding: 12px 24px;
    background-color: #111;
    color: white;
    border-radius: 8px;
    cursor: pointer;
    transition: 0.3s ease;

    &:hover {
      background-color: #FF5722;
    }
  }
}

/* Responsive */
@media (max-width: 768px) {
  .produkdetail {
    flex-direction: column;

    &__image {
      margin-bottom: 20px;
    }
  }
}
```

- Buat komponen detail product views/DetailProduct/index.tsx

```
my-app > src > view > DetailProduct > index.tsx > default
import { ProductType } from "@/type/product.type";
import styles from "../DetailProduct/detailProduct.module.scss";

const DetailProduct = ({products}: {products: ProductType}) => {
    return (
        <div className={styles.produkdetail}>
            <div className={styles.produkdetail}>
                <div className={styles.produkdetail__image}>
                    <img src={products.image} alt={products.name}/>
                </div>
                <div className={styles.produkdetail__info}>
                    <h1 className={styles.produkdetail__name}>{products.name}</h1>
                    <p className={styles.produkdetail__category}>
                        {products.category}
                    </p>
                    <p className={styles.produkdetail__price}>
                        Rp {products.price.toLocaleString()}
                    </p>
                    <p className={styles.produkdetail__size}>
                        Size: {products.size}
                    </p>
                </div>
            </div>
        </div>
    );
}
export default DetailProduct;
```

- Modifikas pages/product/[product].tsx

```
my-app > src > pages > produk > [id].tsx > ...
import fetcher from "@/utils/swr/fetcher";
import { useRouter } from "next/router";
import useSWR from "swr";
import DetailProduct from "@/view/DetailProduct";

const HalamanProduk = () => {
    //const router = useRouter();
    //console.log(Router);
    const {query} = useRouter();
    const {data, error,isLoading} = useSWR(`/api/products/${query.id}`, fetcher);
    return (
        <div>
            <DetailProduct products={isLoading ? [] : data.data} />
        </div>
    );
};

export default HalamanProduk;
```

- Modifikasi views/DetailProduct/index.tsx

```
import { ProductType } from "@/type/product.type";
import styles from "../DetailProduct/detailProduct.module.scss";

const DetailProduct = ({products}: {products: ProductType}) => {
    return (
        <>
            <h1>Detal Produk</h1>
            <div className={styles.produkdetail}>
                <div className={styles.produkdetail__image}>
                    <img src={products.image} alt={products.name} />
                </div>
                <div className={styles.produkdetail__info}>
                    <h2 className={styles.produkdetail__name}>{products.name}</h2>
                    <p className={styles.produkdetail__category}>{products.category}</p>
                    <p className={styles.produkdetail__price}>
                        Rp {products.price.toLocaleString()}
                    </p>
                </div>
            </div>
        </>
    );
}
export default DetailProduct;
```

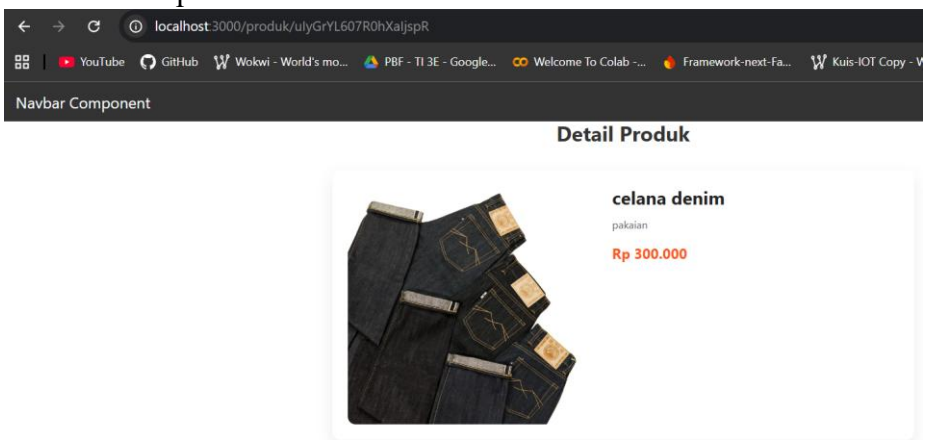
- Merapikan agar judul produk di tengah

```
import { ProductType } from "@/type/product.type";
import styles from "../DetailProduct/detailProduct.module.scss";

const DetailProduk = ({ products }: { products: ProductType }) => {
  return (
    <>
    <h1 className={styles.tittle}>Detail Produk</h1>
    <div className={styles.produkdetail}>
      <div className={styles.produkdetail__image}>
        <img src={products.image} alt={products.name} />
      </div>

      <div className={styles.produkdetail__info}>
        <h1 className={styles.produkdetail__name}>{products.name}</h1>
        <p className={styles.produkdetail__category}>{products.categor
        <p className={styles.produkdetail__price}>
          Rp {products.price.toLocaleString("id-ID")}
        </p>
      </div>
    </div>
    </>
  );
};
```

- Menambahkan styling di file scss

```
.tittle {
  font-size: 32px;
  font-weight: bold;
  text-align: center;
  margin-bottom: 40px;
  color: #333;
}
```

- Hasil tampilan



3. Implementasi SSR

my-app > src > pages > produk > [id].tsx > getServerSideProps

```
import DetailProduct from "@/view/DetailProduct";
import { ProductType } from "@/type/product.type";

const HalamanProduk = ({data} : { data: ProductType }) => {

    return (
        <DetailProduct products={data} />
    )
}

export default HalamanProduk;

export async function getServerSideProps( context: { params: { id: string } }) {
    const { params } = context;
    const id = params?.id;
    const res = await fetch(`http://localhost:3000/api/produk/${id}`);
    const json = await res.json();

    return { props: {data: json.data } };
}
```

4. Implementasi Static Site Generation (Dynamic SSG)

```
import DetailProduct from "@/view/DetailProduct";
import { ProductType } from "@/type/product.type";

const HalamanProduk = ({data} : { data: ProductType }) => {

    return (
        <DetailProduct products={data} />
    )
}

export default HalamanProduk;

export async function getStaticPaths() {
    const res = await fetch("http://localhost:3000/api/produk");
    const responses : { data: ProductType[] } = await res.json();

    const paths = responses.data.map((product: ProductType) => ({
        params: { id: product.id }
    }));

    return {
        paths,
        fallback: false
    };
}

export async function getStaticProps({ params }: { params: { id: string } }) {
    const res = await fetch(`http://localhost:3000/api/produk/${params.id}`);
    const data : { data: ProductType[] } = await res.json();

    return {
        props: {
            data : data.data
        }
    };
}
```

```
my-app > src > view > DetailProduct > index.tsx > DetailProduk
import { ProductType } from "@/type/product.type";
import styles from "../DetailProduct/detailProduct.module.scss";

const DetailProduk = ({ products }: { products: ProductType }) => {
  return (
    <>
    <h1 className={styles.tittle}>Detail Produk</h1>
    <div className={styles.produkdetail}>
      <div className={styles.produkdetail__image}>
        <img src={products.image && products.image} alt={products.name} />
      </div>

      <div className={styles.produkdetail__info}>
        <h1 className={styles.produkdetail__name}>{products.name}</h1>
        <p className={styles.produkdetail__category}>{products.category}</p>
        <p className={styles.produkdetail__price}>
          Rp {products.price && products.price.toLocaleString("id-ID")}
        </p>
      </div>
    </div>
    </>
  );
};

export default DetailProduk;
```

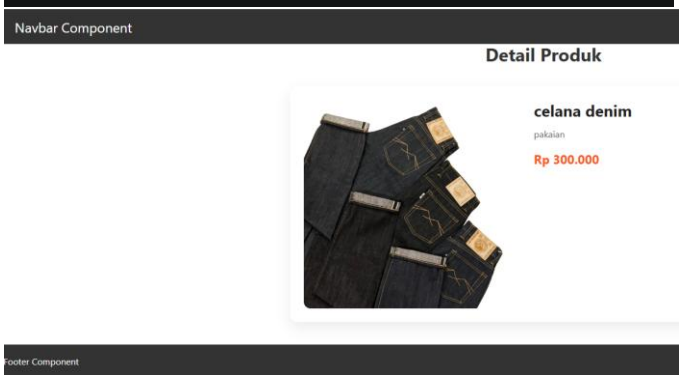
Tugas Praktikum

1. Buat 3 halaman:

- CSG
- SSR
- SSG

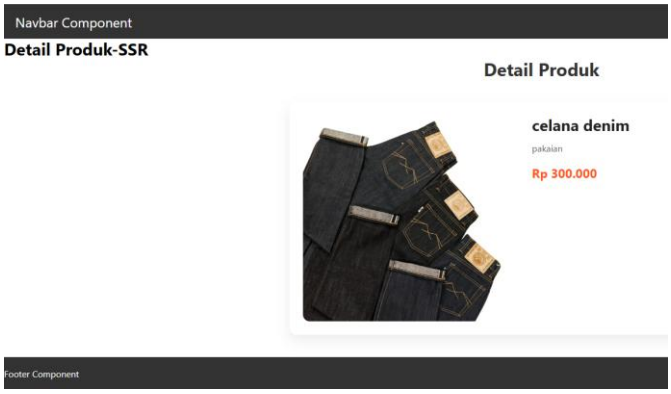
CSR:

```
my-app > src > pages > produk > [id].tsx > HalamanProduk
import { useRouter } from "next/router";
import fetcher from "@/utils/swr/fetcher";
import useSWR from "swr";
import DetailProduct from "@/view/DetailProduct";
import { ProductType } from "@/type/product.type";

const HalamanProduk = () => {

    const {query} = useRouter();
    const productParam = query.id;
    const productId = Array.isArray(productParam) ? productParam[0] : productParam;

    const { data, error, isLoading } = useSWR<{ data: ProductType }>(
        productId ? `/api/produk/${productId}` : null,
        fetcher,
    );

    if (!productId) {
        return <div>Loading...</div>;
    }

    if (error) {
        return <div>Gagal memuat data produk.</div>;
    }

    if (!data || isLoading) {
        return <div>Loading...</div>;
    }

    return (
        <DetailProduct products={data.data} />
    )
}

export default HalamanProduk;
```



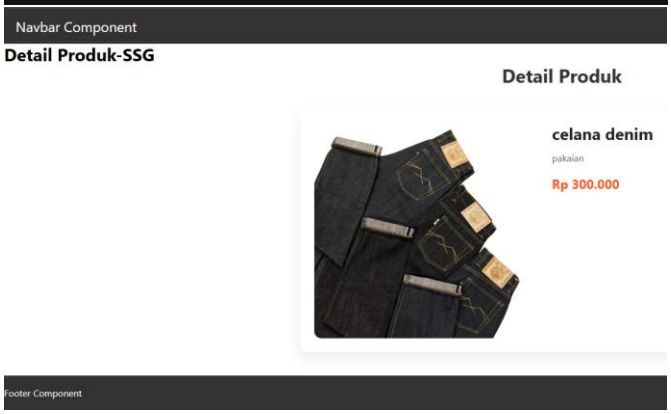
SSR:

```
my-app > src > pages > produk > server > [id].tsx > getServerSideProps
import DetailProduct from "@/view/DetailProduct";
import { ProductType } from "@/type/product.type";

const HalamanProdukServer = ({data} : { data: ProductType }) => {

    return (
        <div>
            <h1>Detail Produk-SSR</h1>
            <DetailProduct products={data} />
        </div>
    )
}

export default HalamanProdukServer;

export async function getServerSideProps( context: { params: { id: string } }) {
    const { params } = context;
    const id = params?.id;
    const res = await fetch(`http://localhost:3000/api/produk/${id}`);
    const json = await res.json();

    return { props: {data: json.data } };
}
```



SSG:

```
import DetailProduct from "@/view/DetailProduct";
import { ProductType } from "@/type/product.type";

const HalamanProdukStatic = ({data} : { data: ProductType }) => {

    return (
        <div>
            <h1>Detail Produk-SSG</h1>
            <DetailProduct products={data} />
        </div>
    )
}

export default HalamanProdukStatic;

export async function getStaticPaths() {
    const res = await fetch("http://localhost:3000/api/produk");
    const responses : { data: ProductType[] } = await res.json();

    const paths = responses.data.map((product: ProductType) => ({
        params: { id: product.id }
    }));

    return {
        paths,
        fallback: false
    };
}

export async function getStaticProps({ params }: { params: { id: string } }) {
    const res = await fetch(`http://localhost:3000/api/produk/${params.id}`);
    const data : { data: ProductType[] } = await res.json();

    return {
        props: {
            data : data.data
        }
    };
}
```



2. **Buat tabel perbandingan:**

Aspek	CSR	SSR	SSG
Loading	lambat, navigasi selanjutnya cepat	cepat, HTML sudah jadi dari server	Paling cepat, file statis di-cache CDN
Build Required	Satu kali build saat deployment	Tidak perlu build ulang, server aktif tiap request	Wajib build ulang setiap ada perubahan konten
SEO	Kurang optimal, crawler sulit baca konten JS	Sangat baik, konten lengkap saat di-crawl	Sangat baik, halaman statis mudah diindeks
Perubahan Data	Dinamis, data diambil dari API setelah halaman dimuat	Dinamis, data diambil server tiap ada akses	Statis, konten berubah hanya saat build ulang

Pertanyaan Refleksi

1. Mengapa getStaticPaths wajib pada dynamic SSG?

Jawab: getStaticPaths wajib digunakan untuk mendaftarkan daftar ID atau parameter rute yang akan dibuatkan halaman statisnya secara otomatis pada saat proses build

2. Mengapa CSR membutuhkan loading state?

Jawab: CSR membutuhkan loading state karena proses pengambilan data dilakukan oleh browser melalui API setelah halaman dimuat. Berbeda dengan SSR, di mana data sudah tersedia bahkan sebelum halaman ditampilkan ke pengguna

3. Mengapa SSG tidak menampilkan produk baru tanpa build ulang?

Jawab: Hal ini terjadi karena pada metode SSG, aplikasi hanya mengambil data dan menghasilkan file HTML statis sekali saja pada saat proses build. Produk baru yang ditambahkan ke database setelah proses tersebut tidak akan dikenali hingga sistem dijalankan ulang (build)

4. Mana metode terbaik untuk halaman detail e-commerce?

Jawab: SSR (Server Side Rendering) adalah metode terbaik untuk detail e-commerce karena data (seperti stok dan harga) selalu diambil setiap kali halaman diakses sehingga selalu akurat , sekaligus tetap memberikan performa SEO yang optimal

5. Apa risiko menggunakan SSG untuk produk yang sering berubah?

Jawab: Risiko utamanya adalah menampilkan data yang usang atau tidak akurat (seperti harga lama atau stok yang sebenarnya sudah habis) kepada pengguna, karena halaman tidak akan memperbarui kontennya secara otomatis saat database berubah